

Experiment 1: Linear Search

```
n = int(input("Enter number of elements: "))  
a = list(map(int, input("Enter elements: ").split()))  
key = int(input("Enter key: "))  
  
for i in range(n):  
    if a[i] == key:  
        print("Key found at index", i)  
        break  
else:  
    print("Key not found")
```

Online C Compiler

```
1 #include <stdio.h>  
2  
3 int main() {  
4     int a[] = {10, 25, 30, 45, 50};  
5     int key = 30, n = 5, i;  
6  
7     for(i = 0; i < n; i++) {  
8         if(a[i] == key) {  
9             printf("Key found at index %d", i);  
10            return 0;  
11        }  
12    }  
13    printf("Key not found");  
14    return 0;  
15 }  
16 }
```

Run

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.64 Mb

Your Output

Key found at index 2

Experiment 2: Binary Search (Recursive)

```
def binary_search(a, low, high, key):
```

```
    if low > high:
```

```
        return -1
```

```
    mid = (low + high) // 2
```

```
    if a[mid] == key:
```

```
        return mid
```

```
    elif key < a[mid]:
```

```
        return binary_search(a, low, mid - 1, key)
```

```
    else:
```

```
        return binary_search(a, mid + 1, high, key)
```

```
a = list(map(int, input("Enter sorted elements: ").split()))
```

```
key = int(input("Enter key: "))
```

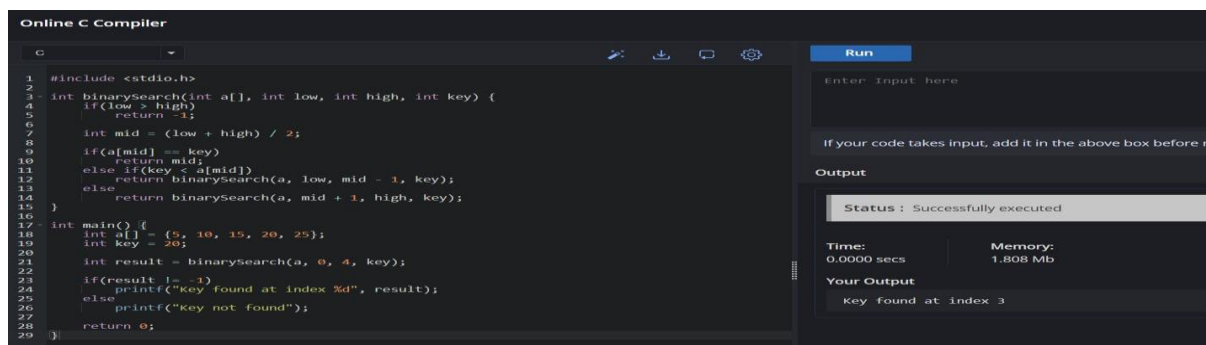
```
result = binary_search(a, 0, len(a) - 1, key)
```

```
if result != -1:
```

```
    print("Key found at index", result)
```

```
else:
```

```
    print("Key not found")
```



The screenshot shows an online C compiler interface. The code editor on the left contains a C program that implements a recursive binary search. The program defines a `binarysearch` function that takes an array `a`, low and high indices, and a `key`. It uses recursion to find the index of the `key` in the sorted array. The `main` function initializes an array `a` with values {5, 10, 15, 20, 25}, sets `key` to 20, and calls `binarysearch`. It then prints the result. The right panel shows the execution status as 'Successfully executed' with a time of 0.0000 secs and memory usage of 1.808 Mb. The output shows 'Key found at index 3'.

```
Online C Compiler
```

```
1 #include <stdio.h>
2
3 int binarysearch(int a[], int low, int high, int key) {
4     if(low > high)
5         return -1;
6
7     int mid = (low + high) / 2;
8
9     if(a[mid] == key)
10        return mid;
11    else if(key < a[mid])
12        return binarysearch(a, low, mid - 1, key);
13    else
14        return binarysearch(a, mid + 1, high, key);
15 }
16
17 int main() {
18     int a[] = {5, 10, 15, 20, 25};
19     int key = 20;
20
21     int result = binarysearch(a, 0, 4, key);
22
23     if(result != -1)
24         printf("Key found at index %d", result);
25     else
26         printf("Key not found");
27
28     return 0;
29 }
```

Run

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.808 Mb

Your Output

Key found at index 3

Experiment 3: Factorial – Iterative and Recursive

```
def factorial_recursive(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    return n * factorial_recursive(n - 1)
```

```
n = int(input("Enter n: "))
```

```
fact = 1
```

```
for i in range(1, n + 1):
```

```
    fact = fact * i
```

```
print("Iterative Factorial =", fact)
```

```
print("Recursive Factorial =", factorial_recursive(n))
```

The screenshot shows an online C compiler interface. The code editor contains a C program that defines a recursive function `factorialRecursive` and a `main` function. The `main` function sets `n = 5` and `fact = 1`, then uses a loop to calculate the iterative factorial, followed by a call to the recursive function. The program is executed, and the output panel shows the status as 'Successfully executed' with a time of 0.0000 secs and memory usage of 1.764 Mb. The final output displays 'Iterative Factorial = 120' and 'Recursive Factorial = 120'.

```
1 #include <stdio.h>
2
3 int factorialRecursive(int n) {
4     if(n == 0)
5         return 1;
6     return n * factorialRecursive(n - 1);
7 }
8
9 int main() {
10     int n = 5, i;
11     int fact = 1;
12
13     // Iterative
14     for(i = 1; i <= n; i++)
15         fact = fact * i;
16
17     printf("Iterative Factorial = %d\n", fact);
18
19     // Recursive
20     printf("Recursive Factorial = %d", factorialRecursive(n));
21
22     return 0;
23 }
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.764 Mb

Your Output

Iterative Factorial = 120
Recursive Factorial = 120

Experiment 4: Fibonacci Series

```
def fibonacci(n):
```

```
    if n <= 1:
```

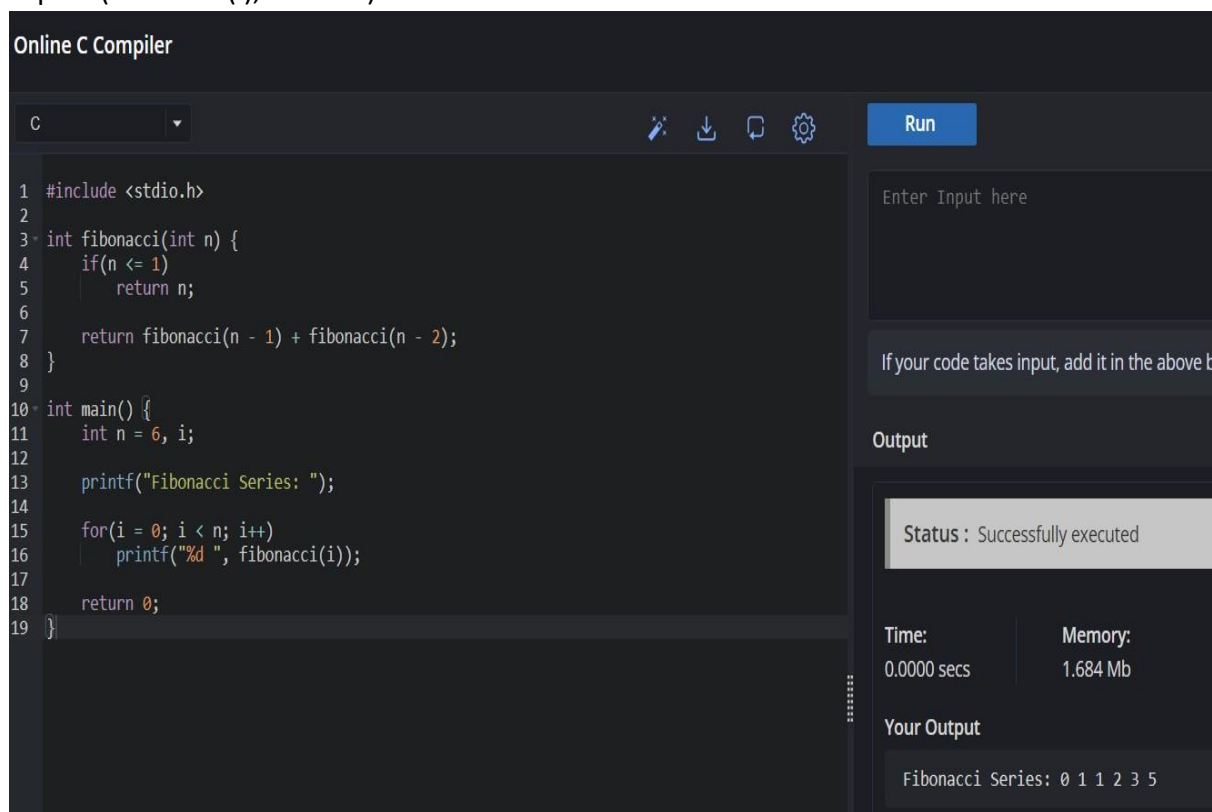
```
        return n
```

```
    return fibonacci(n - 1) + fibonacci(n - 2)
```

```
n = int(input("Enter number of terms: "))
```

```
for i in range(n):
```

```
    print(fibonacci(i), end=" ")
```



The screenshot shows an online C compiler interface. The code editor on the left contains the following C code:

```
1 #include <stdio.h>
2
3 int fibonacci(int n) {
4     if(n <= 1)
5         return n;
6
7     return fibonacci(n - 1) + fibonacci(n - 2);
8 }
9
10 int main() {
11     int n = 6, i;
12
13     printf("Fibonacci Series: ");
14
15     for(i = 0; i < n; i++)
16         printf("%d ", fibonacci(i));
17
18     return 0;
19 }
```

On the right side, there is a 'Run' button. Below it, the 'Output' section shows the status 'Status : Successfully executed'. Below that, the 'Your Output' section displays the result: 'Fibonacci Series: 0 1 1 2 3 5'. Above the output, there is a text input field labeled 'Enter Input here' and a note: 'If your code takes input, add it in the above b'.

Experiment 5: Matrix Multiplication

```
r1 = int(input("Enter rows of first matrix: "))
```

```
c1 = int(input("Enter columns of first matrix: "))
```

```
A = []
```

```
print("Enter first matrix:")
```

```
for i in range(r1):
```

```
    A.append(list(map(int, input().split())))
```

```
r2 = int(input("Enter rows of second matrix: "))
```

```
c2 = int(input("Enter columns of second matrix: "))
```

```
B = []
```

```
print("Enter second matrix:")
```

```
for i in range(r2):
```

```
    B.append(list(map(int, input().split())))
```

```
C = [[0] * c2 for _ in range(r1)]
```

```
for i in range(r1):
```

```
    for j in range(c2):
```

```
        for k in range(c1):
```

```
            C[i][j] += A[i][k] * B[k][j]
```

```
print("Result:")
```

```
for row in C:
```

```
    print(row)
```

```
1  #include <stdio.h>
2
3  void merge(int a[], int low, int mid, int high) {
4      int temp[50];
5      int i = low, j = mid + 1, k = 0;
6
7      while(i <= mid && j <= high) {
8          if(a[i] < a[j])
9              temp[k++] = a[i++];
10         else
11             temp[k++] = a[j++];
12     }
13
14     while(i <= mid)
15         temp[k++] = a[i++];
16
17     while(j <= high)
18         temp[k++] = a[j++];
19
20     for(i = low, k = 0; i <= high; i++, k++)
21         a[i] = temp[k];
22 }
23
24 void mergeSort(int a[], int low, int high) {
25     if(low < high) {
26         int mid = (low + high) / 2;
27
28         mergeSort(a, low, mid);
29         mergeSort(a, mid + 1, high);
30         merge(a, low, mid, high);
31     }
32 }
33
34 int main() {
35     int a[] = {38, 27, 43, 3, 9, 82, 10};
36     int n = 7, i;
37
38     mergeSort(a, 0, n - 1);
39
40     printf("Sorted Array: ");
41
42     for(i = 0; i < n; i++)
43         printf("%d ", a[i]);
```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time:
0.0000 secs

Memory:
1.708 Mb

Your Output

Sorted Array: 3 9 10 27 38 43 82

Experiment 6: Merge Sort

```
def merge_sort(a):
```

```
    if len(a) <= 1:
```

```
        return a
```

```
    mid = len(a) // 2
```

```
    left = merge_sort(a[:mid])
```

```
    right = merge_sort(a[mid:])
```

```
    result = []
```

```
    i = j = 0
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i] < right[j]:
```

```
            result.append(left[i])
```

```
            i += 1
```

```
        else:
```

```
            result.append(right[j])
```

```
            j += 1
```

```
    result += left[i:]
```

```
    result += right[j:]
```

```
    return result
```

```
a = list(map(int, input("Enter elements: ").split()))
```

```
print("Sorted Array:", merge_sort(a))
```

```
1 #include <stdio.h>
2
3 int main() {
4     int A[2][2] = {{1,2},{3,4}};
5     int B[2][2] = {{5,6},{7,8}};
6     int C[2][2] = {0};
7     int i, j, k;
8
9     for(i = 0; i < 2; i++) {
10         for(j = 0; j < 2; j++) {
11             for(k = 0; k < 2; k++) {
12                 C[i][j] += A[i][k] * B[k][j];
13             }
14         }
15     }
16
17     printf("Result:\n");
18
19     for(i = 0; i < 2; i++) {
20         for(j = 0; j < 2; j++) {
21             printf("%d ", C[i][j]);
22         }
23         printf("\n");
24     }
25
26     return 0;
27 }
```

Run

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.652 Mb

Your Output

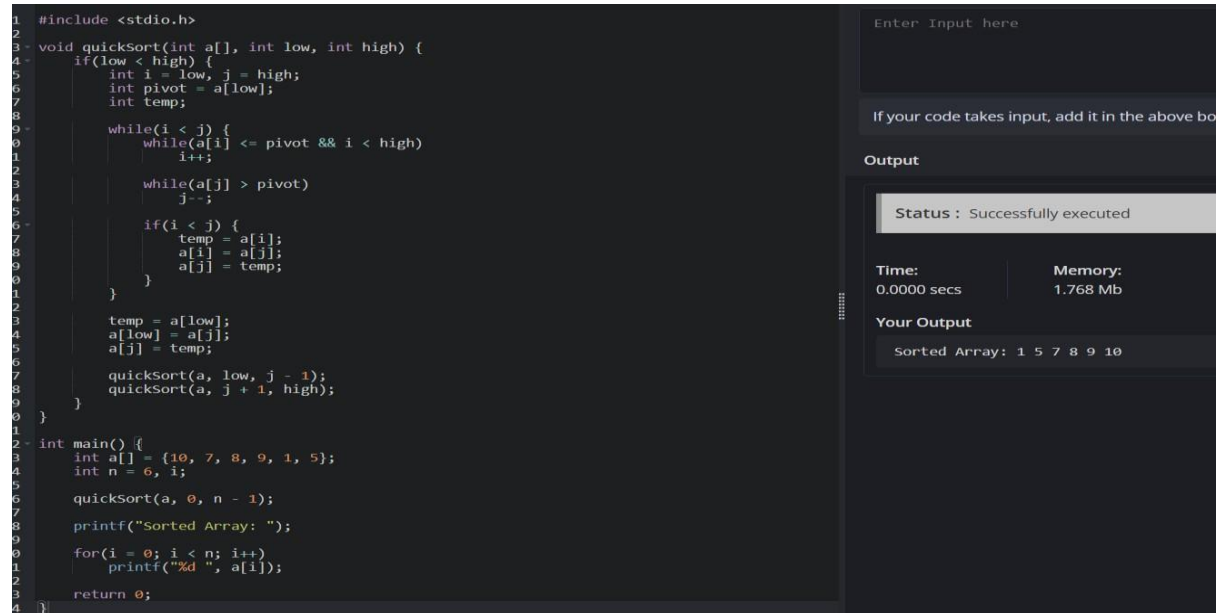
Result:

19 22

43 58

Experiment 7: quick Sort

```
def quick_sort(a):  
    if len(a) <= 1:  
        return a  
  
    pivot = a[0]  
  
    left = [x for x in a[1:] if x <= pivot]  
    right = [x for x in a[1:] if x > pivot]  
  
    return quick_sort(left) + [pivot] + quick_sort(right)  
  
a = list(map(int, input("Enter elements: ").split()))  
  
print("Sorted Array:", quick_sort(a))
```



The screenshot shows a C++ IDE with a dark theme. On the left, a C++ program implements a quicksort algorithm. The code includes `<stdio.h>`, defines `quickSort` with parameters `int a[], int low, int high`, and a `main` function that initializes an array `a = {10, 7, 8, 9, 1, 5}` and calls `quickSort(a, 0, n - 1)`. The output of the program is printed to the console. On the right, a panel shows the execution status: 'Status : Successfully executed'. Below this, a table displays 'Time: 0.0000 secs' and 'Memory: 1.768 Mb'. At the bottom, the 'Your Output' section shows 'Sorted Array: 1 5 7 8 9 10'.

```
1 #include <stdio.h>  
2  
3 void quickSort(int a[], int low, int high) {  
4     if(low < high) {  
5         int i = low, j = high;  
6         int pivot = a[low];  
7         int temp;  
8  
9         while(i < j) {  
10            while(a[i] <= pivot && i < high)  
11                i++;  
12  
13            while(a[j] > pivot)  
14                j--;  
15  
16            if(i < j) {  
17                temp = a[i];  
18                a[i] = a[j];  
19                a[j] = temp;  
20            }  
21  
22            temp = a[low];  
23            a[low] = a[j];  
24            a[j] = temp;  
25  
26            quickSort(a, low, j - 1);  
27            quickSort(a, j + 1, high);  
28        }  
29    }  
30  
31 int main() {  
32     int a[] = {10, 7, 8, 9, 1, 5};  
33     int n = 6, i;  
34  
35     quickSort(a, 0, n - 1);  
36  
37     printf("Sorted Array: ");  
38  
39     for(i = 0; i < n; i++)  
40         printf("%d ", a[i]);  
41  
42     return 0;  
43 }
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.768 Mb

Your Output

Sorted Array: 1 5 7 8 9 10

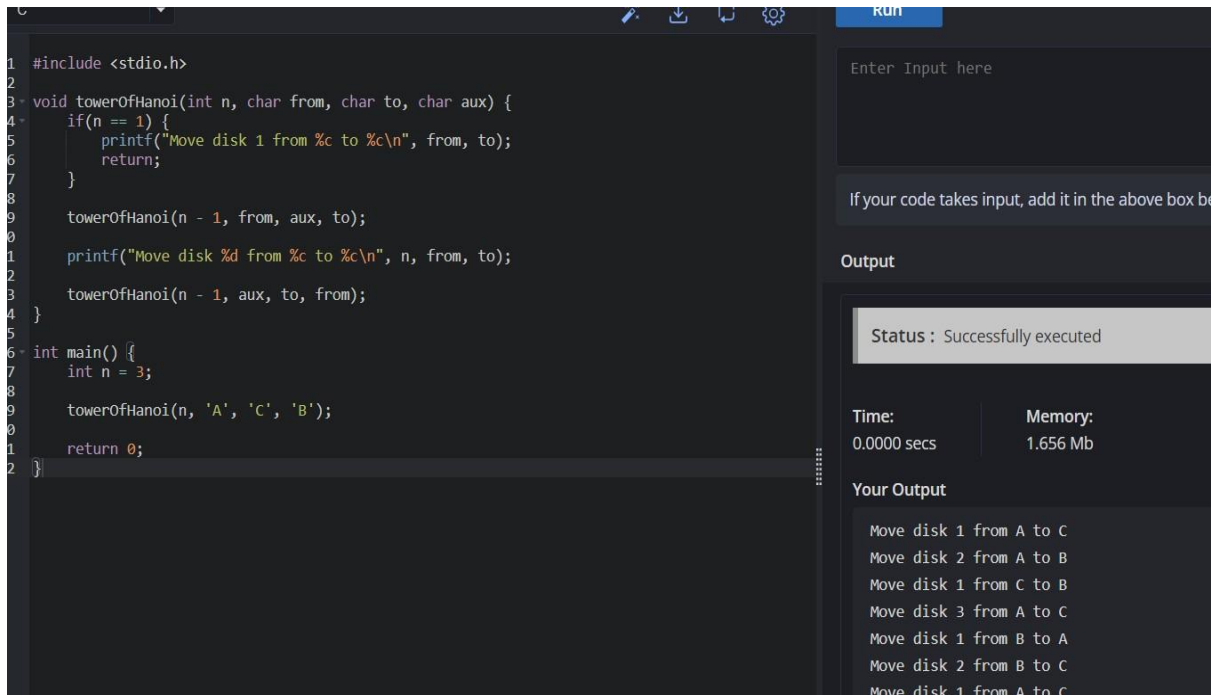
Experiment 8: Tower of Hanoi

```
def tower_of_hanoi(n, source, destination, auxiliary):  
  
    if n == 1:  
  
        print("Move disk 1 from", source, "to", destination)  
  
        return  
  
    tower_of_hanoi(n - 1, source, auxiliary, destination)  
  
    print("Move disk", n, "from", source, "to", destination)
```

```
tower_of_hanoi(n - 1, auxiliary, destination, source)
```

```
n = int(input("Enter number of disks: "))
```

```
tower_of_hanoi(n, 'A', 'C', 'B')
```



The screenshot shows a C++ IDE with a dark theme. The code for the Tower of Hanoi problem is written in C++ and is being executed. The code is as follows:

```
1 #include <stdio.h>
2
3 void towerOfHanoi(int n, char from, char to, char aux) {
4     if(n == 1) {
5         printf("Move disk 1 from %c to %c\n", from, to);
6         return;
7     }
8     towerOfHanoi(n - 1, from, aux, to);
9     printf("Move disk %d from %c to %c\n", n, from, to);
10    towerOfHanoi(n - 1, aux, to, from);
11 }
12
13 int main() {
14     int n = 3;
15     towerOfHanoi(n, 'A', 'C', 'B');
16     return 0;
17 }
```

The IDE has a "Run" button in the top right corner. Below the code editor, there is an "Output" panel. The output panel shows the status "Status : Successfully executed" and the execution time and memory usage:

Time:	Memory:
0.0000 secs	1.656 Mb

Below the execution details, there is a "Your Output" section showing the output of the program:

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

9.gcd and lcm

```
def gcd(a, b):
```

```
    if b == 0:
```

```
        return a
```

```
    return gcd(b, a % b)
```

```
a, b = map(int, input("Enter two numbers: ").split())
```

```
print("GCD =", gcd(a, b))
```

```
C
1 #include <stdio.h>
2
3 int gcd(int a, int b) {
4     if(b == 0)
5         return a;
6
7     return gcd(b, a % b);
8 }
9
10 int main() {
11     int a = 48, b = 18;
12
13     printf("GCD = %d", gcd(a, b));
14
15     return 0;
16 }
```

Run

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.68 Mb

Your Output

GCD = 6

10.insertion sort

```
a = list(map(int, input("Enter elements: ").split()))
```

```
for i in range(1, len(a)):
```

```
    key = a[i]
```

```
    j = i - 1
```

```
    while j >= 0 and a[j] > key:
```

```
        a[j + 1] = a[j]
```

```
        j -= 1
```

```
    a[j + 1] = key
```

```
print("Sorted Array:", a)
```

```
Online C Compiler
C
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {12, 11, 13, 5, 6};
5     int n = 5;
6     int i, j, key;
7
8     for(i = 1; i < n; i++) {
9         key = a[i];
10        j = i - 1;
11
12        while(j >= 0 && a[j] > key) {
13            a[j + 1] = a[j];
14            j--;
15        }
16        a[j + 1] = key;
17    }
18
19    printf("Sorted Array: ");
20
21    for(i = 0; i < n; i++)
22        printf("%d ", a[i]);
23
24    return 0;
25 }
26 }
```

Run

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.68 Mb

Your Output

Sorted Array: 5 6 11 12 13

11.Heap sort

```
import heapq
```

```
a = list(map(int, input("Enter elements: ").split()))
```

```
heapq.heapify(a)
```

```
sorted_array = []
```

```
while a:
```

```
    sorted_array.append(heapq.heappop(a))
```

```
print("Sorted Array:", sorted_array)
```

```
1  #include <stdio.h>
2
3  void heapify(int a[], int n, int i) {
4      int largest = i;
5      int left = 2 * i + 1;
6      int right = 2 * i + 2;
7      int temp;
8
9      if(left < n && a[left] > a[largest])
10         largest = left;
11
12     if(right < n && a[right] > a[largest])
13         largest = right;
14
15     if(largest != i) {
16         temp = a[i];
17         a[i] = a[largest];
18         a[largest] = temp;
19         heapify(a, n, largest);
20     }
21 }
22
23 void heapSort(int a[], int n) {
24     int i, temp;
25
26     for(i = n / 2 - 1; i >= 0; i--)
27         heapify(a, n, i);
28
29     for(i = n - 1; i > 0; i--) {
30         temp = a[0];
31         a[0] = a[i];
32         a[i] = temp;
33         heapify(a, i, 0);
34     }
35 }
36
37
38
39 int main() {
40     int a[] = {4, 10, 3, 5, 1};
41     int n = 5, i;
42
43     heapSort(a, n);
44 }
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.764 Mb

Your Output

Sorted Array: 1 3 4 5 10

12.counting inversion

```
a = list(map(int, input("Enter elements: ").split()))
```

```
count = 0
```

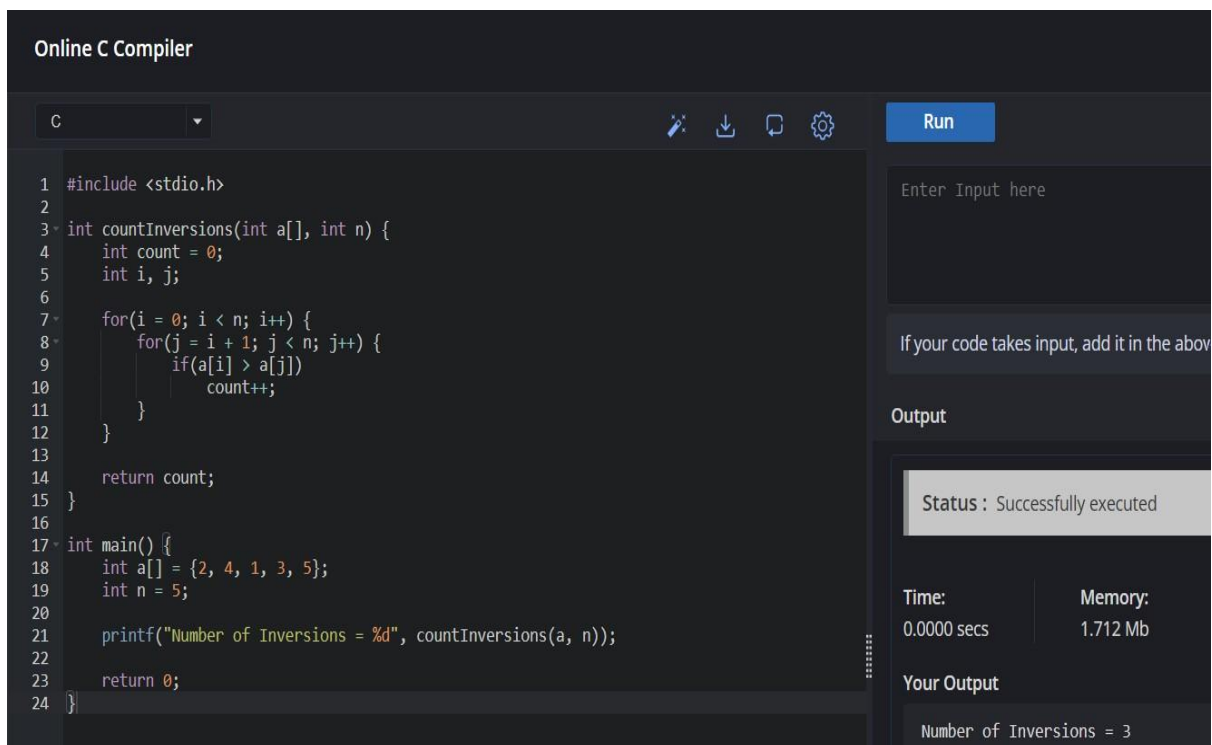
```
for i in range(len(a)):
```

```
    for j in range(i + 1, len(a)):
```

```
        if a[i] > a[j]:
```

```
            count += 1
```

```
print("Number of Inversions =", count)
```



The screenshot shows an online C compiler interface. The code editor on the left contains a C program that defines a function `countInversions` and a `main` function. The `main` function uses the array `{2, 4, 1, 3, 5}` and prints the result of `countInversions(a, 5)`. The right sidebar shows the execution status as 'Successfully executed', with a time of 0.0000 secs and memory usage of 1.712 Mb. The output section displays 'Number of Inversions = 3'.

```
Online C Compiler
```

C

```
1 #include <stdio.h>
2
3 int countInversions(int a[], int n) {
4     int count = 0;
5     int i, j;
6
7     for(i = 0; i < n; i++) {
8         for(j = i + 1; j < n; j++) {
9             if(a[i] > a[j])
10                count++;
11         }
12     }
13
14     return count;
15 }
16
17 int main() {
18     int a[] = {2, 4, 1, 3, 5};
19     int n = 5;
20
21     printf("Number of Inversions = %d", countInversions(a, n));
22
23     return 0;
24 }
```

Run

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.712 Mb

Your Output

Number of Inversions = 3

Experiment 13: Maximum Subarray Sum – Kadane's Algorithm

```
a = list(map(int, input("Enter elements: ").split()))
```

```
current = a[0]
```

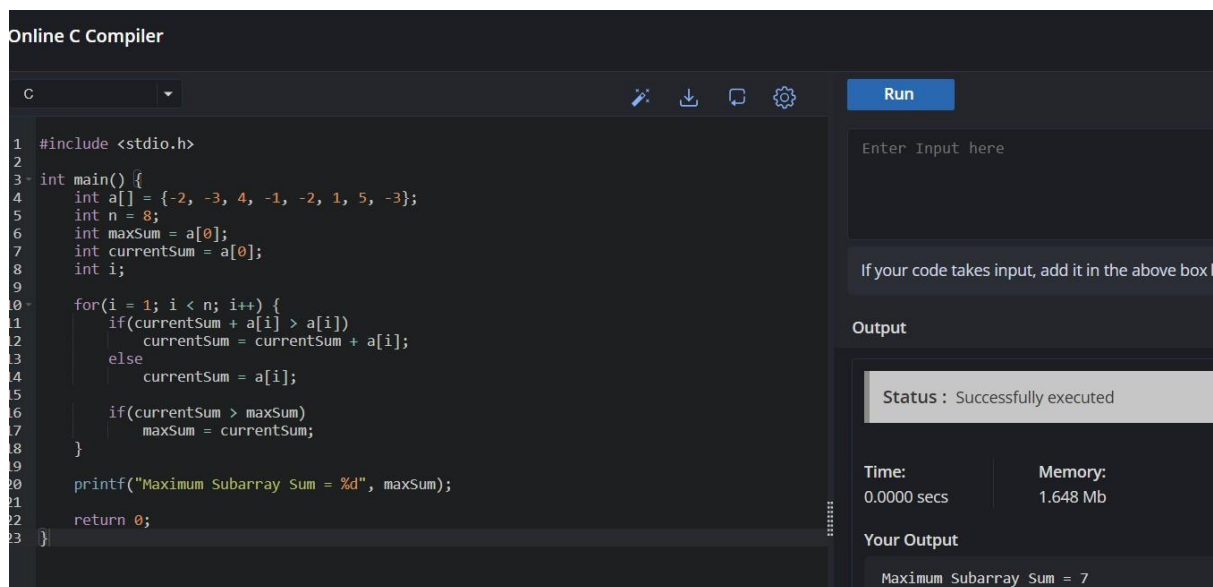
```
maximum = a[0]
```

```
for i in range(1, len(a)):
```

```
    current = max(a[i], current + a[i])
```

```
    maximum = max(maximum, current)
```

```
print("Maximum Subarray Sum =", maximum)
```



The screenshot shows an online C compiler interface. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int a[] = {-2, -3, 4, -1, -2, 1, 5, -3};
5     int n = 8;
6     int maxSum = a[0];
7     int currentSum = a[0];
8     int i;
9
10    for(i = 1; i < n; i++) {
11        if(currentSum + a[i] > a[i])
12            currentSum = currentSum + a[i];
13        else
14            currentSum = a[i];
15
16        if(currentSum > maxSum)
17            maxSum = currentSum;
18    }
19
20    printf("Maximum Subarray Sum = %d", maxSum);
21
22    return 0;
23 }
```

The right-hand side of the interface shows the execution status and output:

- Run** button
- Input field: "Enter Input here"
- Instruction: "If your code takes input, add it in the above box"
- Output** section:
 - Status: Successfully executed
 - Time: 0.0000 secs
 - Memory: 1.648 Mb
 - Your Output: Maximum Subarray Sum = 7

14: Exponentiation – Fast Power

```
def power(x, n):
```

```
    if n == 0:
```

```
        return 1
```

```
    if n % 2 == 0:
```

```
        half = power(x, n // 2)
```

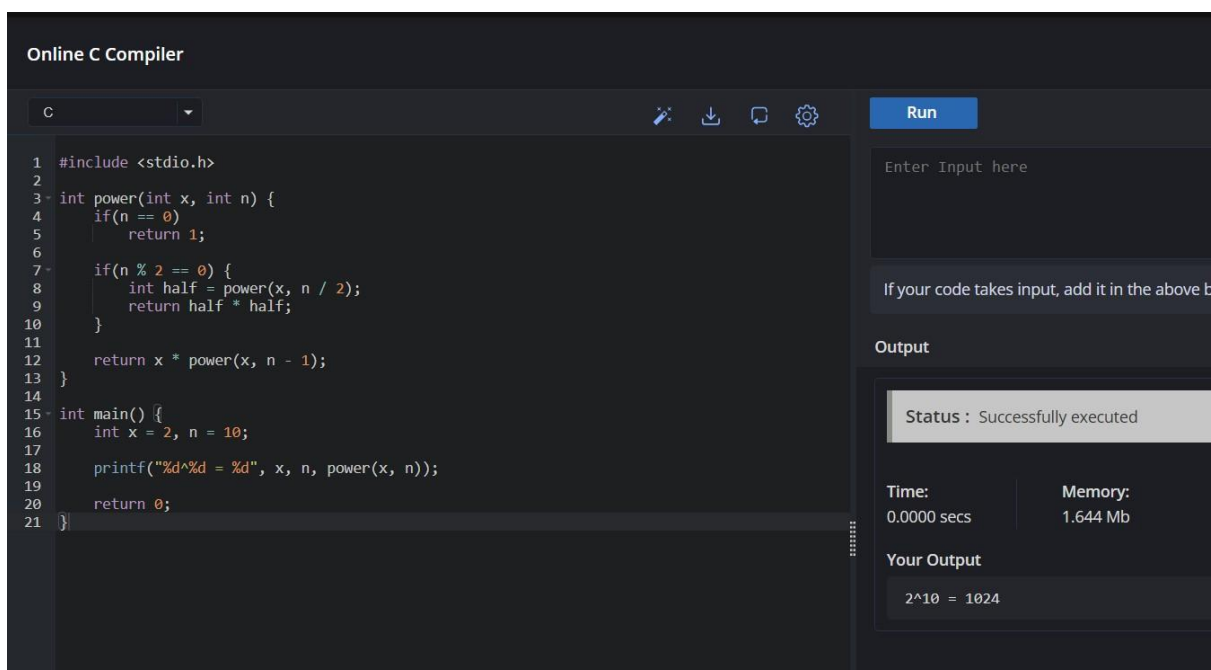
```
        return half * half
```

```
    return x * power(x, n - 1)
```

```
x = int(input("Enter base: "))
```

```
n = int(input("Enter exponent: "))
```

```
print("Answer =", power(x, n))
```



The screenshot shows an online C compiler interface. The code editor on the left contains the following C code:

```
1 #include <stdio.h>
2
3 int power(int x, int n) {
4     if(n == 0)
5         return 1;
6
7     if(n % 2 == 0) {
8         int half = power(x, n / 2);
9         return half * half;
10    }
11
12    return x * power(x, n - 1);
13 }
14
15 int main() {
16     int x = 2, n = 10;
17     printf("%d^%d = %d", x, n, power(x, n));
18
19     return 0;
20 }
```

The right sidebar shows the execution results:

- Run** button
- Enter Input here
- If your code takes input, add it in the above box
- Output** section:
- Status**: Successfully executed
- Time**: 0.0000 secs
- Memory**: 1.644 Mb
- Your Output**: 2^10 = 1024

15.closest pair of elements

```
import math

n = int(input("Enter number of points: "))

points = []

print("Enter points:")
for i in range(n):
    x, y = map(int, input().split())
    points.append((x, y))

minimum = float('inf')
p1 = p2 = None

for i in range(n):
    for j in range(i + 1, n):
        distance = math.sqrt(
            (points[i][0] - points[j][0]) ** 2 +
            (points[i][1] - points[j][1]) ** 2
        )

        if distance < minimum:
            minimum = distance
            p1 = points[i]
            p2 = points[j]

print("Closest Pair:", p1, p2)
print("Distance =", round(minimum, 2))
```

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      int points[4][2] = {
6          {1, 2},
7          {4, 5},
8          {7, 8},
9          {3, 1}
10     };
11
12     double minDistance = 999999;
13     int p1, p2;
14     int i, j;
15
16     for(i = 0; i < 4; i++) {
17         for(j = i + 1; j < 4; j++) {
18             double dx = points[i][0] - points[j][0];
19             double dy = points[i][1] - points[j][1];
20
21             double distance = sqrt(dx * dx + dy * dy);
22
23             if(distance < minDistance) {
24                 minDistance = distance;
25                 p1 = i;
26                 p2 = j;
27             }
28         }
29     }
30
31     printf("Closest Pair: (%d,%d) and (%d,%d)\n",
32           points[p1][0], points[p1][1],
33           points[p2][0], points[p2][1]);
34
35     printf("Distance = %.2f", minDistance);
36
37     return 0;
38 }

```

Enter Input here

If your code takes input, add it in the ab

Output

Status : Successfully executed

Time:

0.0000 secs

Memory:

2.256 Mb

Your Output

Closest Pair: (1,2) and (3,1)

Distance = 2.24